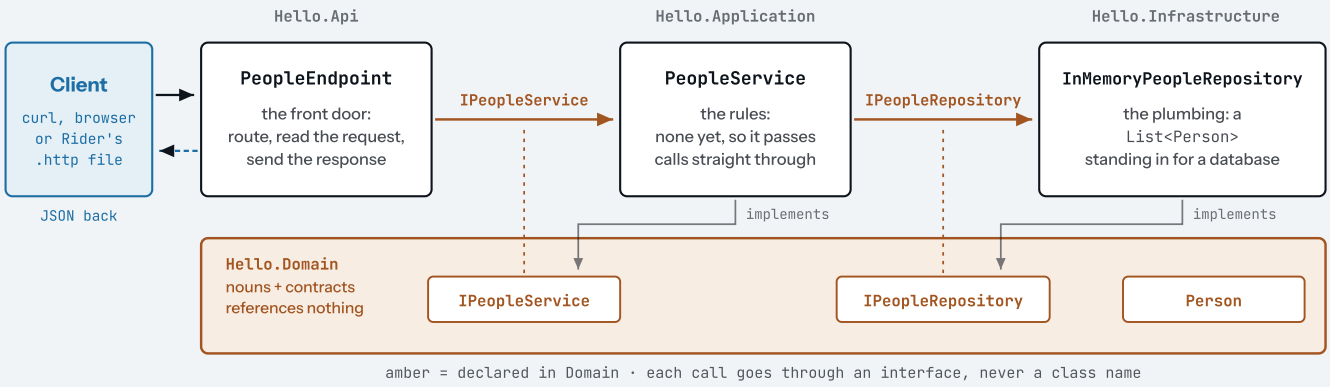
Build a Basic API

Week 1 read an existing service. Week 2 built one from an empty folder: **Hello**, a .NET 10 API with three endpoints, four projects and an in-memory list of people. This sheet is the **recipe to rebuild it**, with the C# you meet on the way.

> HOMEWORK Rebuild Hello without the recording, using the order at the bottom of this page, then add one endpoint of your own.

START HERE What we built

ONE REQUEST, EVERY LAYER



Each layer knows only the interface in front of it. `PeopleEndpoint` asks for an `IPeopleService` and never names `PeopleService`. Swapping the list for a real database is one new class in Infrastructure and one changed line in `Program.cs`: Week 1's *inversion*, at its smallest.

The three endpoints

REAL RESPONSES FROM THE FINISHED API

REQUEST	ENDPOINT CLASS · BASE CLASS	RESPONSE
`GET /hello?name=Elly`	`HelloEndpoint` · `Endpoint<HelloRequest, string>`	`"Hello, Elly"`
`GET /people`	`PeopleEndpoint` · `EndpointWithoutRequest<IEnumerable<Person>>`	`[{"name":"John","age":20}, {"name":"Jane","age":21}]` C# Name → JSON name: camelCase both ways
`POST /people`	`AddPersonEndpoint` · `Endpoint<AddPersonRequest, string>` body: `{"name":"Mary","age":30}`	`"Person added"`

Rebuild order

GET IT RUNNING, THEN BUILD INSIDE-OUT

#	DO	DONE WHEN	SEE
1	New **ASP.NET Core Web API** on .NET 10. Run it untouched.	`/weatherforecast → 200`	§01
2	Install FastEndpoints, write `HelloEndpoint`, delete the weather sample.	`/hello?name=Elly → "Hello, Elly"`	§02
3	Add Application, Domain, Infrastructure as **class libraries**; add references.	`dotnet build → 4 projects`	§04
4	Domain: `Person`, `IPeopleService`, `IPeopleRepository`.	`builds`	§04
5	`PeopleService` in Application; `InMemoryPeopleRepository` in Infrastructure.	`builds`	§05
6	Register both in `Program.cs` with `AddSingleton`.	`app starts`	§05
7	`PeopleEndpoint`, `AddPersonEndpoint`. **Restart** to pick them up.	`POST Mary · GET → 3 people`	§08

01 Template to running

REBUILD STEP 1

In Rider: **New Solution → ASP.NET Core Web API**, `net10.0`, C#. Or the CLI, which also makes the `.gitignore` to add before your first commit:

```
dotnet new sln -n Hello           # Hello.slnx: workspace root (.slnx is new in .NET 10)
dotnet new webapi -n Hello.Api     # the only project that runs
dotnet sln add Hello.Api
dotnet new gitignore               # ignores bin/ and obj/, the build output
dotnet run --project Hello.Api     # then GET http://localhost:<port>/weatherforecast
```

`Program.cs` is the whole app, and it has two halves. Everything before `Build()` registers what exists. Everything after it sets up what happens to each request.

THE TEMPLATE, TRIMMED

```
var builder = WebApplication.CreateBuilder(args);
// 1 · REGISTER: what exists (the container)
builder.Services.AddOpenApi();

var app = builder.Build();

// 2 · PIPELINE: what each request passes through
if (app.Environment.IsDevelopment())
    app.MapOpenApi();
app.UseHttpsRedirection();
app.MapGet("/weatherforecast", () => { ... });
app.Run();

record WeatherForecast(DateOnly Date,
    int TemperatureC, string? Summary);
```

WHAT EACH PART IS

`Add…()`
Puts a service in the container. Order mostly doesn't matter. ≈ *wrapping your app in providers.*

`Use…()`
Adds middleware: a step every request passes through, in the order written. ≈ *Express* `app.use()`.

`MapGet()`
A **minimal API** endpoint: a route plus a lambda, defined in `Program.cs`. ≈ *Express* `app.get('/x', fn)`.

`MapOpenApi()`
Serves a generated JSON description of every endpoint at `/openapi/v1.json`, in Development only. The template has no Swagger UI page.

`/` **returns 404** because only `/weatherforecast` is mapped. The weather is random, made up on every call. The `record` at the bottom is explained in §03.

02 Switching to FastEndpoints

REBUILD STEP 2

Minimal APIs pile every route into `Program.cs`. **FastEndpoints**, a NuGet package, gives each endpoint its own class instead: request type, handler and response type together. `user.service` uses MVC controllers, which group many endpoints into one class.

INSTALL (V8.3) AND REGISTER · BOTH LINES NEEDED

```
dotnet add Hello.Api package FastEndpoints

// Program.cs, register half
builder.Services.AddFastEndpoints();

// Program.cs, pipeline half
app.UseFastEndpoints();
// ↑ finds every endpoint class in the
//   project, once, at startup
```

Or Rider's NuGet window. Packages must support your target framework, so upgrading .NET means upgrading them too.

ENDPOINTS/HELLOENDPOINT.CS

```
public class HelloRequest
{
    public string Name { get; set; } = "";
}

public class HelloEndpoint
    : Endpoint<HelloRequest, string>
{
    public override void Configure()
    {
        Get("/hello");
        AllowAnonymous();
    }

    public override async Task HandleAsync(
        HelloRequest req, CancellationToken ct)
    {
        await Send.OkAsync($"Hello, {req.Name}", ct);
    }
}
```

PIECE	WHAT IT DOES	FRONT-END EQUIVALENT
`Endpoint<TReq, TRes>`	The base class. The two type arguments (**generics**) fix what comes in and what goes out, and the compiler holds the handler to both.	`Handler<Req, Res>` in TS
`Configure()`	Runs **once at startup**: HTTP verb, route, auth rules.	the route definition
`HandleAsync()`	Runs **on every request**, with the request already bound from query, route and body.	the handler function
`Send.OkAsync(x, ct)`	Writes status 200 and `x` as JSON.	`res.json(x)`
`CancellationToken`	Fires if the client disconnects. Pass it to anything slow.	`AbortSignal`
`AllowAnonymous()`	FastEndpoints endpoints **require auth by default**. Leave this out with no auth set up and every call returns 500.	`[AllowAnonymous]` on a controller

02 Pick the base class by its shape

FOUR VARIANTS

BASE CLASS	REQUEST	RESPONSE	USED IN HELLO FOR
`Endpoint<TReq, TRes>`	typed	typed	`GET /hello`, `POST /people`
`EndpointWithoutRequest<TRes>`	none	typed	`GET /people`, a plain "give me the list"
`Endpoint<TReq>`	typed	untyped	not used; for endpoints that return no body
`EndpointWithoutRequest`	none	untyped	not used; a ping or health check

The request has to be a **class**. The binder fills properties by name (`?name=Elly` goes into `Name`), and a `string` has no properties to fill. Mark's first attempt hit this; it's the third error below.

DEBUGGING The errors hit live

STARTUP → PROGRAM.CS / CONFIGURE() · REQUEST → HANDLEASYNC()

WHEN	YOU SEE	CAUSE → FIX
startup	`No service for type 'FastEndpoints.IServiceResolver' has been registered.`	You called `UseFastEndpoints()` without `AddFastEndpoints()`. Add the register-half line.
startup	`No HTTP Verbs declared on: [Hello.Api.Endpoints.HelloEndpoint]`	`Configure()` is empty or missing. Add `Get("/hello")`.
request	`200 · "Hello, "`	The request type is `string`, so nothing binds. Use a request class with a `Name` property.
request	`405 Method Not Allowed`	You added a `POST /people` endpoint while the app was still running. Only `GET /people` was registered at startup, so the path exists but not that verb. **Restart.**
request	`500 · contains authorization metadata, but a middleware was not found that supports authorization`	No `AllowAnonymous()`. Not hit live, but you will hit it.

03 C# words met in passing

WITH THE TYPESCRIPT YOU ALREADY KNOW

KIND	WHAT IT IS	NEAREST TS	IN HELLO
`class`	A blueprint. Variables hold a *reference*, so two variables can point at one object.	`class`	`Person`, `PeopleService`
`interface`	A contract with no code. A class must *declare* it: `: IPeopleService`. A matching shape isn't enough (Week 1).	`interface`, but opt-in	`IPeopleService`
`record`	A class with short syntax and **value equality**: two records holding the same values are equal. Positional properties can't be reassigned. `record Point(int X, int Y); new Point(1,2) == new Point(1,2) → true`	no equivalent; a readonly type compared field by field	the template's `WeatherForecast`
`struct`	A **value type**: assigning it copies it, and it can't be null.	how `number` behaves	`int`, `DateOnly`
`enum`	A named set of constants.	`enum`	`DayOfWeek.Monday`

Who can see it

`public`	Anyone, in any project. The class's public contract.
`internal`	Only code in the **same project**. **The default** for a class with no modifier.
`protected`	The class and its subclasses.
`private`	Only the class itself: its internals.

Easy to miss with four projects. Write `class Person` without `public` and it is `internal` to Domain. Api then fails to build with `CS0122: 'Person' is inaccessible due to its protection level`. Rider's templates add `public`; hand-typed classes don't.

Inheriting from a base class

`abstract`	Can't be created with `new`; only subclassed. `Endpoint<,>` is abstract.
`virtual`	A base method that subclasses *may* replace. `Configure` and `HandleAsync` are virtual.
`override`	Replaces the base version completely.
`base.X()`	Calls the base version from inside your override. ≈ *`super.x()`*

Asked in the session: after an override, does the base `Configure()` still run? **No.** It runs only if you call `base.Configure()` yourself, and FastEndpoints doesn't need you to. `static` means something belongs to the type, not an instance. Endpoints aren't static, so the framework creates instances of them.

04 Splitting into four projects

REBUILD STEPS 3–4 · RECAP: WEEK 1 "THE FOUR PROJECTS"

One project works, but anything can then use anything. Splitting it gives each layer its own compiled project, and the references between them become the rules. Week 1 covers why; this is how.

CREATE AND REFERENCE

```
dotnet new classlib -n Hello.Application
dotnet new classlib -n Hello.Domain
dotnet new classlib -n Hello.Infrastructure
dotnet sln add Hello.Application Hello.Domain \
               Hello.Infrastructure

# "X reference Y" = X is allowed to use Y
dotnet add Hello.Api reference Hello.Application \
               Hello.Domain Hello.Infrastructure
dotnet add Hello.Application reference Hello.Domain
dotnet add Hello.Infrastructure reference Hello.Domain
```

Delete each `Class1.cs`. In Rider: **Add** → **New Project** → **Class Library**, then **Add** → **Reference** on the project.

THE FINISHED TREE

```
Hello.slnx
Hello.Api/                  Sdk.Web · runs
  Program.cs  appsettings.json
  Properties/launchSettings.json
  Endpoints/HelloEndpoint.cs
            PeopleEndpoint.cs
            AddPersonEndpoint.cs
Hello.Application/          Sdk · library
  Services/PeopleService.cs
Hello.Domain/               Sdk · references nothing
  Entities/Person.cs
  Interfaces/IPeopleService.cs
             IPeopleRepository.cs
Hello.Infrastructure/       Sdk · library
  Repositories/InMemoryPeopleRepository.cs
```

The trap Mark hit: three web projects. Made from the wrong template, Application, Domain and Infrastructure each got their own `Program.cs`, `appsettings.json` and `launchSettings.json`. Only Api runs, so only Api should have them. Check the first line of each `.csproj`: `Sdk="Microsoft.NET.Sdk.Web"` for Api, plain `Microsoft.NET.Sdk` for the rest.

HELLO.DOMAIN · THE NOUNS

```
namespace Hello.Domain.Entities;  // project + folder

public class Person
{
    public string Name { get; set; } = "";
    public int Age { get; set; }
}
```

HELLO.DOMAIN · THE CONTRACTS

```
namespace Hello.Domain.Interfaces;

public interface IPeopleRepository
{
    Task<IEnumerable<Person>> GetPeople();
    Task AddPerson(Person person);
}
// IPeopleService: the same two methods
```

A namespace follows the folder path, and `using Hello.Domain.Entities;` is the import. **Interfaces:** Hello puts both in Domain. Many codebases keep a service interface like `IPeopleService` in Application, since only Api calls it. `IPeopleRepository` must be in Domain: Infrastructure implements it, and here Infrastructure can't see Application.

05 Wiring it together

REBUILD STEPS 5-6 · RECAP: WEEK 1 "WHO BUILDS THESE OBJECTS?"

Each class lists what it needs in its constructor, and the container supplies it. Two ways to write that, which compile to almost the same thing:

CLASSIC · A BACKING FIELD

```
public class PeopleService : IPeopleService
{
    private readonly IPeopleRepository _repo;

    public PeopleService(IPeopleRepository repo)
    {
        _repo = repo;
    }

    public Task<IEnumerable<Person>> GetPeople()
        => _repo.GetPeople();
}
```

PRIMARY CONSTRUCTOR · C# 12+, WHAT RIDER SUGGESTS

```
public class PeopleService(IPeopleRepository repo)
    : IPeopleService
{
    public Task<IEnumerable<Person>> GetPeople()
        => repo.GetPeople();

    public Task AddPerson(Person person)
        => repo.AddPerson(person);
}
```

05 readonly, and what it doesn't stop

WIRING, CONTINUED

`_repo = other;`	Outside the constructor: **compile error** `CS0191: A readonly field cannot be assigned to`. The reference is fixed.
`_repo.AddPerson(p);`	**Allowed.** `readonly` fixes which object you point at, not what's inside it. ≈ *a TS `const` holding an array: no reassigning, but `push` still works.*
`repo = other;`	On a primary-constructor parameter: **compiles. Easy to miss:** these parameters aren't readonly, so the shorthand drops the guard. Nobody reassigns injected services in practice, so both styles are fine.

HELLO.INFRASTRUCTURE · THE LIST

```
public class InMemoryPeopleRepository
    : IPeopleRepository
{
    private readonly List<Person> _people =
    [
        new() { Name = "John", Age = 20 },
        new() { Name = "Jane", Age = 21 },
    ];

    public Task<IEnumerable<Person>> GetPeople() =>
        Task.FromResult<IEnumerable<Person>>(_people);

    public Task AddPerson(Person person)
    {
        _people.Add(person);
        return Task.CompletedTask;
    }
}
```

HELLO.API · PROGRAM.CS, REGISTER HALF

```
builder.Services.AddFastEndpoints();

// "when someone asks for the interface,
//  give them this class"
builder.Services.AddSingleton<
    IPeopleRepository, InMemoryPeopleRepository>();
builder.Services.AddSingleton<
    IPeopleService, PeopleService>();
```

Api is the only project that references all the others, so it's the only place that can name both the interface and the class. This is the single line you'd change to swap in a database.

The repository isn't really async. It returns `Task` anyway, so a database version can drop in without changing any caller.

LIFETIMES Why the repository has to be a Singleton

OBJECTS = LISTS

Singleton + Singleton

```
AddSingleton<IPeopleRepository, …>
AddSingleton<IPeopleService, …>
```

One repository for the life of the process. POST Mary, then GET:

`John · Jane · Mary`

Restart the app and Mary is gone. The list was never saved anywhere.

Scoped + Scoped

```
AddScoped<IPeopleRepository, …>
AddScoped<IPeopleService, …>
```

A new repository, and so a new list, for every request. The POST adds Mary to a list that's thrown away when its response goes out:

`"Person added" · then John · Jane`

Scoped repo + Singleton service

```
AddScoped<IPeopleRepository, …>
AddSingleton<IPeopleService, …>
```

In Development the app **refuses to start**:

```
Cannot consume scoped service
'IPeopleRepository' from singleton
'IPeopleService'
```

This is the captive dependency from Week 1.

LIFETIME	THE CONTAINER…	FITS
Singleton	builds one, the first time it's needed, and hands it to everyone until restart	Hello's in-memory list · caches · API clients
Scoped	builds one per HTTP request, shared by everything in that request	a real database repository: the data lives in the database, not in the object
Transient	builds a new one every time anything asks, even twice in one request	an open question from the session; see §09

06 Syntax shortcuts

```
// collection expression (C# 12) ≈ [ ... ]
List<Person> people = [ a, b ];
// target-typed new: type taken from the left
Person p = new() { Name = "Mary" };
// object initialiser: no () needed
new Person { Name = "Mary", Age = 30 };
// interpolation ≈ `Hello, ${name}`
$"Hello, {req.Name}"
// expression body ≈ arrow function
public Task X() => repo.X();
```

Return `IEnumerable<T>` (≈ `Iterable<T>`), not `List<T>`, so the repository can change its storage without changing callers.

07 Translation card

WEEK 2 ROWS

YOU KNOW		HERE IT'S
`app.get('/x', fn)`	→	`app.MapGet / an Endpoint class`
`req.query.name`	→	`req.Name (bound by name)`
`res.json(x)`	→	`Send.OkAsync(x, ct)`
`AbortSignal`	→	`CancellationToken`
`Promise.resolve(x)`	→	`Task.FromResult(x)`
`npm i pkg`	→	`dotnet add package Pkg`
`import … from`	→	`using Namespace;`

Rider: ⌥↩ on a red name: create method, implement members, create derived type, move to file. `ctor` + Tab: a constructor.

08 Adding a feature: POST /people

REBUILD STEP 7 · FIVE PLACES, TOP-DOWN

Once the layers are wired, every feature is the same walk. Mark wrote each call before the method existed, then used ⌥↩ (Alt+Enter on Windows) to create it in the interface and implement it in the class, one layer down at a time.

1	Api	`AddPersonRequest` + `AddPersonEndpoint`
2	Domain	`IPeopleService.AddPerson`
3	App	`PeopleService.AddPerson` → repo
4	Domain	`IPeopleRepository.AddPerson`
5	Infra	`_people.Add(person)`
↻	restart	New endpoint, so restart, or you get a 405.

```
public class AddPersonEndpoint(IPeopleService service)
    : Endpoint<AddPersonRequest, string>
{
    public override void Configure()
    {
        Post("/people");
        AllowAnonymous();
    }

    public override async Task HandleAsync(
        AddPersonRequest req, CancellationToken ct)
    {
        await service.AddPerson(new Person
            { Name = req.Name, Age = req.Age });
        await Send.OkAsync("Person added", ct);
    }
}
```

The refactor at the end: `AddPerson(string name, int age)` became `AddPerson(Person person)`. Rule of thumb: at about four parameters, pass one object instead, as you would in JS.

SHAPE	LIVES IN	IS	WHY IT'S SEPARATE
`Person`	Domain	the **entity**: the thing as the system stores it	grows `Id`, `CreatedAt`… that clients must never set
`AddPersonRequest`	Api	the **request DTO** (data transfer object: properties, no behaviour): exactly what the client may send	it's the public contract; changing it can break callers
`response model`	Api	what goes back: maybe the person plus a message or error codes	Hello returns a bare string for now

09 Homework and what's next

- **Rebuild Hello solo** from the order on page 1, then add an endpoint of your own. A real data source (the Supabase ESL dictionary came up) is a new repository in Infrastructure behind a Domain interface; endpoint and service shouldn't change.
- **Mark's question:** for a stateless helper (say, one that returns `"Hello " + name`), which performs better, Transient or Singleton? Hint: count the objects the container builds over 1,000 requests. And still open: **when would you actually want Transient?**
- **Mentioned only:** CQRS / mediator, where one object receives every request and routes it to its handler. **Planned next:** a lifetimes demo (one of each kind), then a real external service in Infrastructure.